12/08/21

What's the longest program that you have written so far?
A. 0-100
B. 100-1K
C. 1K-10K
D. 10K-.....

Gyan 1: If you want to use a programming language proficiently, then you should have all its technical details on your fingertips.

Environment I am using:
- Python interpreter
- top-level
- REPL : Repeat Evaluate Print Loop

Programming = Data + Processing

If branching
- Python takes indentation seriously.
- Within a code block please use uniform indentation level.

Summary

Arithmetic expressions
int, float, boolean, string
Operators:
  - Arithmetic: +, -, *, / ----> number, number -> number
  - Relational: >, <, ==, <= ... -----> number, number -> boolean
  - Logical: and, or, not ... boolean, boolean -> boolean

Control flow structures:
  If [elif *[else]]
  While

Python is a "DYNAMICALLY TYPED" programming language
- Not 'not strictly'
- Not weakly typed
- Not loosely typed

POLL QUESTION: Variables can be defined in Python without being declared (as in Java, C). Does that make Python a dynamically typed PL? - NO!

Static/dynamic typing of a language is unrelated to whether the language is strictly/strongly typed.
Implicitly typed: absence of type annotations

(strong, weak) X (static, dynamic) X (implicit, explicit).

Type annotation:
int x;
x : int;

17/08/21

Lists are heterogeneously typed.
Lists are mutable.

Python allows aliasing.
Aliasing: The property whereby two references can point to the same object in memory.
This shows us details about the inner workings of a programming language, because aliasing is not always a bad thing, it is a tradeoff between efficiency and security.
Because of aliasing we don't have to keep creating new objects wasting time and memory and instead just point to existing objects using references.
Note that the problem with security is a bigger problem when you have aliasing on mutable objects like lists, for something like strings, tuples, etc. aliasing is not as big a problem as existing data cannot be modified via different references and instead new data can only be added.

SELF STUDY: Slicing in lists

Tuples are immutable. They are safe from the accidents that aliasing can cause to mutable datatypes.

Functions in Python are parametrically polymorphic: Parameters are allowed to be of different types (within the restrictions of typing rules of Python).

Pure/mathematical/stateless functions and impure functions.
In strict terminology, impure functions are not even functions. They are really procedures.

Side-effect -- Imperative programming, side-effects change something either directly or indirectly. Side-effects typically refer to mutating a data structure (changing its value in some way) in memory.
For example,
l1.append(10) => changes the state of the l1 object even though it is not explicitly stated in the append function, so it is a side effect.

range(1, 10) => always returns the same data and doesn't modify any other object/no side-effect.

Pure functions create no effects.

This is why impure functions are more dangerous than pure functions and why designing applications with pure functions as much as possible is more desirable.
A programming paradigm that writes only pure functions is functional programming (FP).

Python is not a pure functional programming language but it does support functional programming.

However a twist to FP is that even taking input or showing output (console or otherwise) is a side-effect, so how can we have pure functional programming. This is why FP is defined to include side-effects but they are to be kept as minimal as possible. Typically, the flow is that inputs are given (side-effect), computations are performed without side-effects and the output is shown (side-effect).

HOMEWORK
Define a function invert(d) which takes a dictionary d and return another which is invert of the dictionary.
Example:
d = { 1 : "one", 2 : "two", 3 : "one" }

invert(d) ---> { "one" : [1,3] "two" : [2] }

23/08/21
Recursive function for computing Factorial
F(N) = 1 if N = 1
= N F(N - 1) otherwise

Recursion is a great method for problem decomposition

Recursive function for computing Fibonacci numbers
Fib(N) = 1 if N = 0, 1
= Fib(N - 1) + Fib(N - 2) otherwise
Write the code for this!

L1 = [1, 2, 3]
L2 = L1

This is a shallow copy as it is just aliasing.
You can avoid this by slicing,
L2 = L1[:]

However this won't work if you have more depth to the object,
L1 = [[1, 2], [3, 4]]
L2 = L1[:]
This is a shallow copy, only the first level is not aliased, however the second level lists are both aliases and changing value of list in L2 will reflect in L1 as well.

So if you want to be truly free of aliasing irrespective of object depth, then you need a deep copy. Here, is a recursive solution for the same,

```
def deep_copy(l):
  ans = []
  for el in l:
      if(type(el) == list):
          ans.append(deep_copy(el))
      else:
          ans.append(el)
  return ans

if __name__ == "__main__":
  print(__name__)
  m1 = [[1, 2], [3, 4]]
  m2 = deep_copy(m1)

  m1[1][1] = 40
  print(m1)
  print(m2)
```

__name__ is a system variable that determines who is running that program, if that program itself is running it then its value is set to __main__
However a program can also be executed by another program, by calling a function that it has imported from that file. For example, deep_copy is imported by another file called client.py, then in that case, if client.py calls deep_copy, the code written in the __main__ portion of deep_copy.py doesn't run, but it would have run if we had not put that code in an if __name__ == "__main__" block.

When we write "import deep_copy" in client.py all the code that is written in deep_copy.py file is executed with __name__ = "deep_copy" and this is why that __main__ block doesn't get executed.

To err is human, to forgive, divine.
To iterate is human, to recurse, divine.

Recursion is one of the pillars of functional programming.
Another is Higher order functions which,
- Take other functions as parameter
and/or
- Return functions as result of computation

List comprehension is syntactic sugar on map + filter.

Using lambda/anonymous functions wherever convenient prevents your namespace from getting cluttered with pointless function names.

In functional programming, functions are first-class objects which,
- Can be passed as parameters to other functions
- Returned as results of computation from other function
- They can be stored in data-structures.

In simpler words, this just means that functions are treated as data.

To demonstrate this, here is a code snippet that uses functions in the 3 ways described above.

```
def add(x, y): return x + y

def make_add(n): return lambda x: add(x, n)

add10 = make_add(10)
# Returns a function add(x, 10), where is a parameter that has to be
provided

add10(5)
# Returns 15 = 5 + 10, x = 5

Lof = [make_add(i) for i in range(1, 11)]
Lof[9](4)
# Returns 14, make_add(10) + 4
```

Sir's attempt to evoke some strong emotions: OOPS is just syntactic sugar over FP.

25/08/21

We don't have to declare class attributes but we typically initialize them at runtime via the constructor method __init__().

This means that we can create attributes dynamically for different objects of the same class.

```
R1 = Rectangle(10, 5)
R2 = Rectangle(5, 10)
R1.x = 5 # Valid
print(R1.x) # Also valid
print(R2.x) # Error: Because x is attribute for R1 object only
```

You can also create class-level data members

```
class Circle:
      pi = 3.141
      ...
      def area(self):
            return Circle.pi * (self.radius ** 2)
# Try to use fully qualified names by using className.classVariableName
instead of just classVariableName or self.classVariableName
```